

Byte Pair Encoding and Linear Text Classification

Assignment 1 CS 6120

Shikha Tiwari¹

¹Khoury College of Computer Sciences, Northeastern University
tiwari.shi@northeastern.edu

Assignment Part 1 Byte Pair Encoding

1. Implement BPE Algorithm-

For implementation of BPE algorithm, I created a class “bytePairEncoding” which had all the relevant function from text pre-processing, encoding/ decoding the text using the learned merge operations. Code is in the file “byte_pair_encoding.py”.

Further , I trained it on sample text and created a bpe_vocab:

```
text = """ The quick brown fox jumps over the lazy dog. The quick brown dog barks loudly.Quick, quick, and quick!
The quick fox is quick. Brown fox, brown dog, brown quick. Lazy dog, lazy fox, lazy brown."""
```

BPE Vocab created using algorithm:

```
['loudly', 'barks', 'loudl', 'brown', 'quick', 'jumps', 'lazy', 'jump', 'brow', 'bark', 'loud', 'over', 'quic', 'jum', 'bro', 'ove',
'bar', 'laz', 'and', 'dog', 'fox', 'the', 'lou', 'qui', 'ju', 'ba', 'br', 'ov', 'do', 'an', 'is', 'fo', 'th', 'la', 'qu', 'lo', 'u', 'a', 'r', 'e', 'g', 'b',
'f', 'w', 't', 'v', 't', 'c', 'k', 'o', 's', 'p', 'n', 'q', 'h', 'i', 'x', 'j', 'm', 'y', 'z', 'd']
```

2. Training on NLTK Dataset-

I utilized NLTK's Gutenberg Corpus, selecting books like "austen-emma.txt," "blake-poems.txt" and "shakespeare-hamlet.txt" for training the BPE algorithm and create a vocabulary based on the training. For training utilized the “bytepairencoding” class methods.

3. Testing on NLTK Dataset-

Evaluates the BPE algorithm on a separate set of books from the NLTK Gutenberg Corpus “austen-persuasion.txt”, “edgeworth-parents.txt”, “whitman-leaves.txt” and used metrics such as coverage analysis, vocab_utiliation to evaluate the defined bpe algorithm.

Output:

```
Book Name: austen-persuasion.txt
Sample tokens ['persu', 'as', 'ion', 'by', 'jane', 'au', 'st', 'en', '1', '8', '1', '8', 'ch', 'ap', 'ter', '1', 'sir', 'wal', 'ter', 'el']
Coverage: 100.0%
Vocabulary Utilization: 94.4015444015444%
```

```
Book Name: edgeworth-parents.txt
Sample tokens ['the', 'par', 'ent', 's', 'ass', 'ist', 'ant', 'by', 'mar', 'i', 'a', 'ed', 'ge', 'wor', 'th', 'the', 'or', 'p', 'ha', 'n']
Coverage: 100.0%
Vocabulary Utilization: 94.98069498069498%
```

```
Book Name: whitman-leaves.txt
Sample tokens ['le', 'ave', 's', 'of', 'gra', 'ss', 'by', 'wal', 't', 'whi', 't', 'man', '1', '8', '5', '5', 'come', 'said', 'my', 'sou']
Coverage: 100.0%
Vocabulary Utilization: 94.5945945945946%
```

Sample tokens for each book:

```
[['persuasion', 'by', 'jane', 'austen', '1818', '],', 'chapter', '1', 'sir', 'walter', 'elliott', ',', 'of', 'kellynch', 'hall', ',', 'in', 'somersetshire', ',']
['the', 'parent', '"', 's', 'assistant', ',', 'by', 'maria', 'edgeworth', '],', 'the', 'orphans', ',', 'near', 'the', 'ruins', 'of', 'the', 'castle', 'of']
['leaves', 'of', 'grass', 'by', 'walt', 'whitman', '1855', '],', 'come', ',', 'said', 'my', 'soul', ',', 'such', 'verses', 'for', 'my', 'body']
```

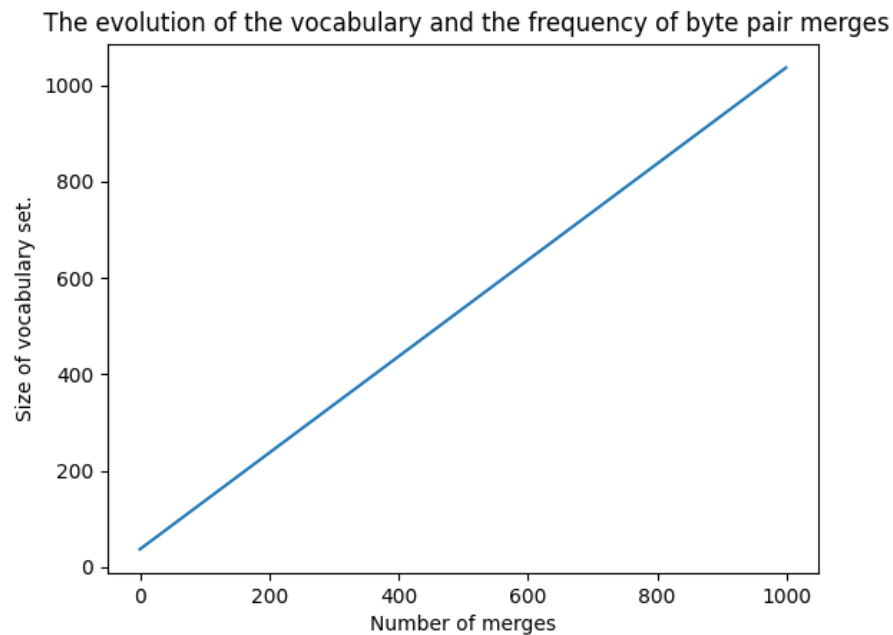
4. Create Reference Tokenization-

Uses NLTK's punkt tokenizer to create a reference tokenization for the test dataset. Save the tokenized results in a json file “reference_tokenizations.json” for further use.

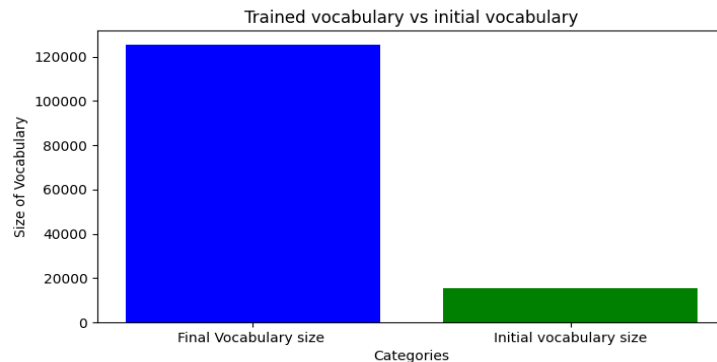
5. Compare with Standard Tokenization -

Implemented metrics to compare bpe algorithm with the standard tokenization method and used- tokenization accuracy, tokenization coverage, precision, recall, f1 score, Jaccard similarity.

6. Visualization-



We could also see from the line chart that the vocabulary size increased linearly with the iterations of merging pairs during the model training phase.



If we compare trained vocabulary vs the initial vocabulary, the trained model has a significantly larger vocabulary size in comparison to the initial dataset for a sample book.

Strengths of BPE:

- **Out-of-Vocabulary Handling:** BPE is effective in dealing with rare or unknown words. By breaking down words into sub word units, it allows the model to understand and generate unseen words during training, improving its generalization.
- **Compression Efficiency:** BPE effectively compresses the vocabulary size by representing frequent character pairs as subword units. This leads to a more compact representation of the text compared to traditional word-based or character-based encoding schemes.

Weaknesses of BPE:

- **Dependency on the Training Corpus:** The quality of the generated sub words heavily depends on the training data. If the training data is not representative of the language's diversity, BPE might not capture all relevant sub words.
- **Fixed Vocabulary:** Once trained, BPE has a fixed vocabulary. Any updates or changes in language use might require retraining or adjusting the BPE model.
- **Training Complexity:** Training BPE requires selecting an appropriate number of merge operations and balancing between under-segmentation (resulting in longer subword units) and over-segmentation (resulting in shorter subword units). Finding the optimal number of merge operations can be challenging and may require experimentation.

Issues with the Implementation and testing:

- **The back and forth process of training and testing the code for various edge cases** such as special characters, empty strings, very large datasets was time consuming.
- **Hyperparameter Tuning:** BPE requires selecting hyperparameters such as the number of merge operations (num_merges) and the maximum vocabulary size. Choosing inappropriate hyperparameters can result in suboptimal tokenization, inefficient vocabulary construction, or excessive memory usage.
- **Data Preprocessing:** Preprocessing steps such as text normalization, punctuation removal, or tokenization can influence the performance of BPE. Inconsistent or incorrect preprocessing may introduce noise, bias, or tokenization errors, leading to inaccurate evaluation results.

Potential Improvements:

- **Parameter Optimization:** Conduct systematic hyperparameter optimization experiments to find the optimal values for parameters such as the number of merge operations (num_merges). Techniques such as grid search or random search can be employed to explore the hyperparameter space efficiently.
- **Data Augmentation:** Augment the training data with diverse and representative examples to improve the robustness and generalization of the BPE model. This can help address issues related to data quality and bias, leading to more reliable tokenization results.

Assignment Part 2 Linear Text Classification

1. Data Preparation-

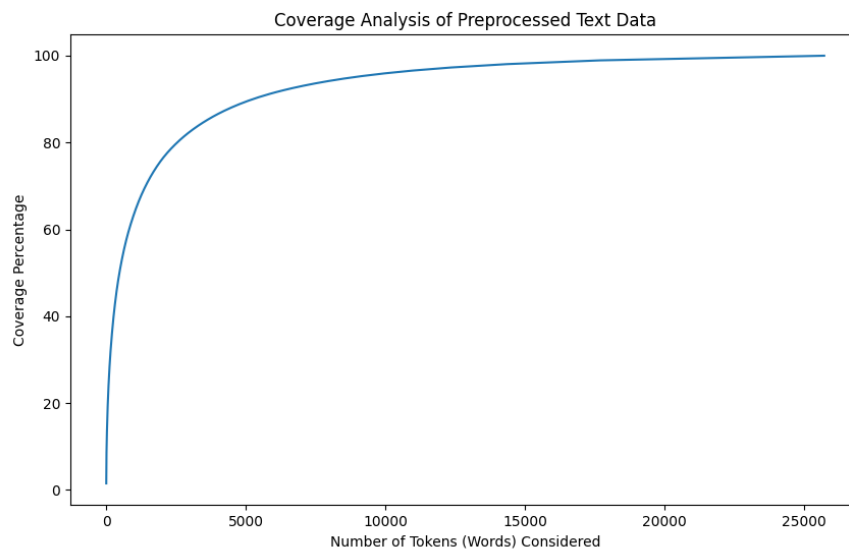
Loaded the movie reviews data and the sample review and corresponding label:

Sample Review: ['""', 'contact', '""', 'shows', 'why', 'jodie', 'foster', 'is', 'one', 'of', 'our', 'best', 'actresses', ':", 'see', 'contact', 'with', 'your', 'family', ':', 'it', 'is', 'a', 'movie', 'for', 'all', 'ages', ':', '---', 'a', 'review', 'by', 'stuart', 'cracraft', 'contact', 'starring', ':', 'jodie', 'foster', ':', 'matthew', 'mcconaughey', ':', 'john', 'hurt', 'director', ':', 'robert', 'zemeckis', 'star']

Label: pos

Then, I preprocessed the data and tokenized the words then performed stemming and removed stop words from the review.

2. Coverage Analysis-



1. How does the coverage change with the number of tokens considered?

We can observe the coverage increases as number of tokens increases however, rate of increase in coverage slow down over time as the model converges and fewer new subword units are discovered. Additionally, the coverage may plateau or reach a saturation point where further increases in the number of tokens have diminishing returns in improving coverage.

2. At what point does the coverage seem to stabilize?

At 10000 tokens, the coverage stabilizes. This indicates that most common roots are already covered, and adding more unique words doesn't significantly increase the coverage.

3. Are there diminishing returns in terms of coverage as the number of tokens increases?

Beyond a certain point, there is a stagnation in coverage percentage. This means that as more unique and less frequent words are considered, they contribute less to increasing the coverage since they are less likely to have a common stem with words in the processed dataset.

Rationalization for Vocabulary Choice :

1. The trade-off between a larger vocabulary (more words) and computational efficiency.

Larger vocabularies might capture more information particularly for rare or out-of-vocabulary terms but can memory usage and longer processing times during vocabulary construction and tokenization and may include many low-frequency words that don't contribute much to the model's performance. On the other hand, prioritizing computational efficiency with a smaller vocabulary reduces memory overhead and processing time but may compromise representation power and coverage.

2. The impact of rare or very common words on the model's generalization.

Rare Words: can be more informative and improve the scope of the model but including too many rare words can lead to overfitting, where the model learns patterns specific to the training data that don't generalize well.

Common Words: Common words (stop words) often carry less informative content for certain tasks. We can remove them as we observed from coverage analysis even if we remove more common words it still capture majority of the information.

3. The need to balance informativeness and model complexity.

Informativeness: A model with a rich vocabulary can capture more detailed information but may become complex. Complexity: A complex model can be more challenging to train and fine-tune. It also risks overfitting if the training data is not sufficiently large or diverse.

4. Any specific considerations for the chosen algorithms (Naive Bayes, Logistic Regression, MLP) in terms of vocabulary size.

For Naive Bayes, Logistic Regression, and Multilayer Perceptron (MLP) algorithms, vocabulary size considerations are crucial. Naive Bayes relies on vocabulary features, where a larger vocabulary can lead to increased sparsity and computational complexity, requiring efficient feature selection or dimensionality reduction techniques. Logistic Regression performs well with large vocabularies but may suffer from increased training time and overfitting if not properly regularized. MLPs can handle large vocabularies but require careful tuning of network architecture and regularization to prevent overfitting and ensure efficient training. In summary, balancing vocabulary size with model complexity and computational resources is essential for optimizing the performance of these algorithms.

3. Algorithm Implementation-

Implemented all three models - Naïve Bayes, Logistic Regression and MLP Classifier. For this, I utilized `make_pipeline` method of `sklearn` and then used "`CountVectorizer`" and "`TfidfVectorizer`". Also, for MLP I used two different models - model 1 with one hidden layer of 100 neurons and model 2 with two hidden layers with 100 and 50 neurons.

4. Evaluation of models -

Naive Bayes with TF:
Accuracy: 0.84
True Positive Rate (TPR): 0.797979797979798
False Positive Rate (FPR): 0.1188118811881188

Naive Bayes with TF-IDF:
Accuracy: 0.8475
True Positive Rate (TPR): 0.787878787878788
False Positive Rate (FPR): 0.09405940594059406

Logistic Regression with TF:
Accuracy: 0.815
True Positive Rate (TPR): 0.7575757575757576
False Positive Rate (FPR): 0.12871287128712872

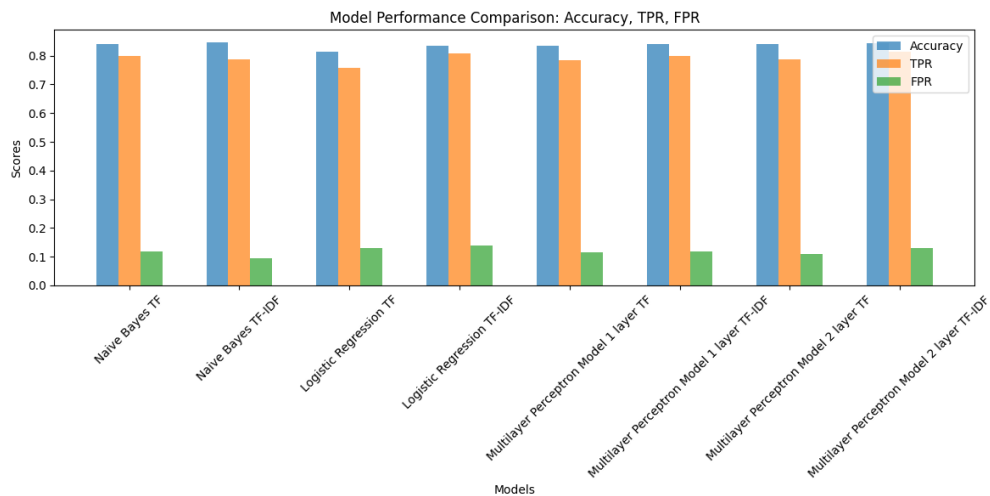
Logistic Regression with TF-IDF:
Accuracy: 0.835
True Positive Rate (TPR): 0.8080808080808081
False Positive Rate (FPR): 0.13861386138613863

MLP with TF (Single layer):
Accuracy: 0.835
True Positive Rate (TPR): 0.7828282828282829
False Positive Rate (FPR): 0.11386138613861387

MLP with TF-IDF (Single layer):
Accuracy: 0.84
True Positive Rate (TPR): 0.797979797979798
False Positive Rate (FPR): 0.1188118811881188

MLP with TF (Two layers):
Accuracy: 0.84
True Positive Rate (TPR): 0.787878787878788
False Positive Rate (FPR): 0.10891089108910891

MLP with TF-IDF (Two layers):
Accuracy: 0.8425
True Positive Rate (TPR): 0.8131313131313131
False Positive Rate (FPR): 0.12871287128712872



- Naive Bayes with TF-IDF emerged as the top-performing model in terms of accuracy and FPR, making it suitable for tasks where minimizing false positives is crucial.
- MLP with TF-IDF (two layers) demonstrated the highest TPR, making it suitable for tasks where correctly identifying positive instances is paramount.
- Logistic Regression models, while generally exhibiting lower performance compared to Naive Bayes and MLP, still showed competitive results and may be preferred for their simplicity and interpretability.

1. Model Comparison -

Naive Bayes:

Accuracy: Naive Bayes achieves an accuracy of 0.84 with TF and 0.8475 with TF-IDF, indicating that it performs relatively well in classifying the data.

True Positive Rate (TPR): The TPR for Naive Bayes is 0.7979 with TF and 0.7879 with TF-IDF, suggesting that it effectively identifies the positive class instances.

False Positive Rate (FPR): The FPR for Naive Bayes is 0.1188 with TF and 0.0941 with TF-IDF, indicating a moderate rate of false positives, especially lower with TF-IDF.

Logistic Regression:

Accuracy: Logistic Regression achieves an accuracy of 0.815 with TF and 0.835 with TF-IDF, demonstrating its effectiveness in classification.

True Positive Rate (TPR): The TPR for Logistic Regression is 0.7576 with TF and 0.8081 with TF-IDF, indicating its ability to identify positive instances, especially improved with TF-IDF.

False Positive Rate (FPR): The FPR for Logistic Regression is 0.1287 with TF and 0.1386 with TF-IDF, suggesting a relatively higher rate of false positives compared to Naive Bayes.

MLP (Single layer):

Accuracy: MLP with a single layer achieves an accuracy of 0.835 with TF and 0.84 with TF-IDF, showing competitive performance.

True Positive Rate (TPR): The TPR for MLP (single layer) is 0.7828 with TF and 0.7980 with TF-IDF, indicating its effectiveness in identifying positive instances.

False Positive Rate (FPR): The FPR for MLP (single layer) is 0.1139 with TF and 0.1188 with TF-IDF, demonstrating relatively low rates of false positives.

MLP (Two layers):

Accuracy: MLP with two layers achieves an accuracy of 0.84 with TF and 0.8425 with TF-IDF, demonstrating comparable performance to other models.

True Positive Rate (TPR): The TPR for MLP (two layers) is 0.7879 with TF and 0.8131 with TF-IDF, indicating its effectiveness in identifying positive instances, especially improved with TF-IDF.

False Positive Rate (FPR): The FPR for MLP (two layers) is 0.1089 with TF and 0.1287 with TF-IDF, showing relatively low rates of false positives, especially with TF.

2. TF vs TFIDF:

Overall, the use of TF-IDF resulted in improved performance across all algorithms, with higher accuracy, higher TPR, and lower FPR compared to TF. Naive Bayes with TF-IDF emerged as the top-performing model, achieving the highest accuracy and lowest FPR among all combinations of algorithms and representations. MLP with TF-IDF also demonstrated competitive performance, particularly in terms of TPR, making it suitable for tasks where correctly identifying positive instances is crucial.

TF (Term Frequency):

- Good for highlighting the frequency of terms within a document. However, it doesn't account for the importance of a term in relation to the entire dataset.
- Might give undue weight to terms that are frequent in a document but not informative overall.

TF-IDF (Term Frequency-Inverse Document Frequency):

- Balances term frequency with how common the word is across all documents. This helps in giving more weight to terms that are important in a particular document but not common in all documents.
- Often leads to better performance in text classification tasks as it reduces the weight of less informative terms.

3. Insights into Strengths and Limitations in the Context of Sentiment Analysis

- Naive Bayes is computationally efficient and effective for large datasets, assumption of feature independence might oversimplify the complexity of sentiment analysis.
- Logistic Regression offers interpretable results and moderate computational requirements can capture linear relationships between features and labels, it may struggle to capture more

complex patterns present in sentiment analysis tasks, limiting its effectiveness compared to more complex models.

- MLP captures complex patterns in data, making them well-suited for sentiment analysis tasks due to its non-linear modeling capabilities. However, MLPs require larger amounts of data and computational resources for training, and there's a higher risk of overfitting due to their increased flexibility and capacity to memorize noise in the data.

When choosing between TF and TF-IDF, the latter is generally more effective for sentiment analysis as it helps the model focus on more relevant features. However, the choice between TF and TF-IDF may depend on the specific characteristics of the dataset, such as the prevalence of rare or common terms and the importance of term frequency versus document frequency in capturing sentiment nuances.